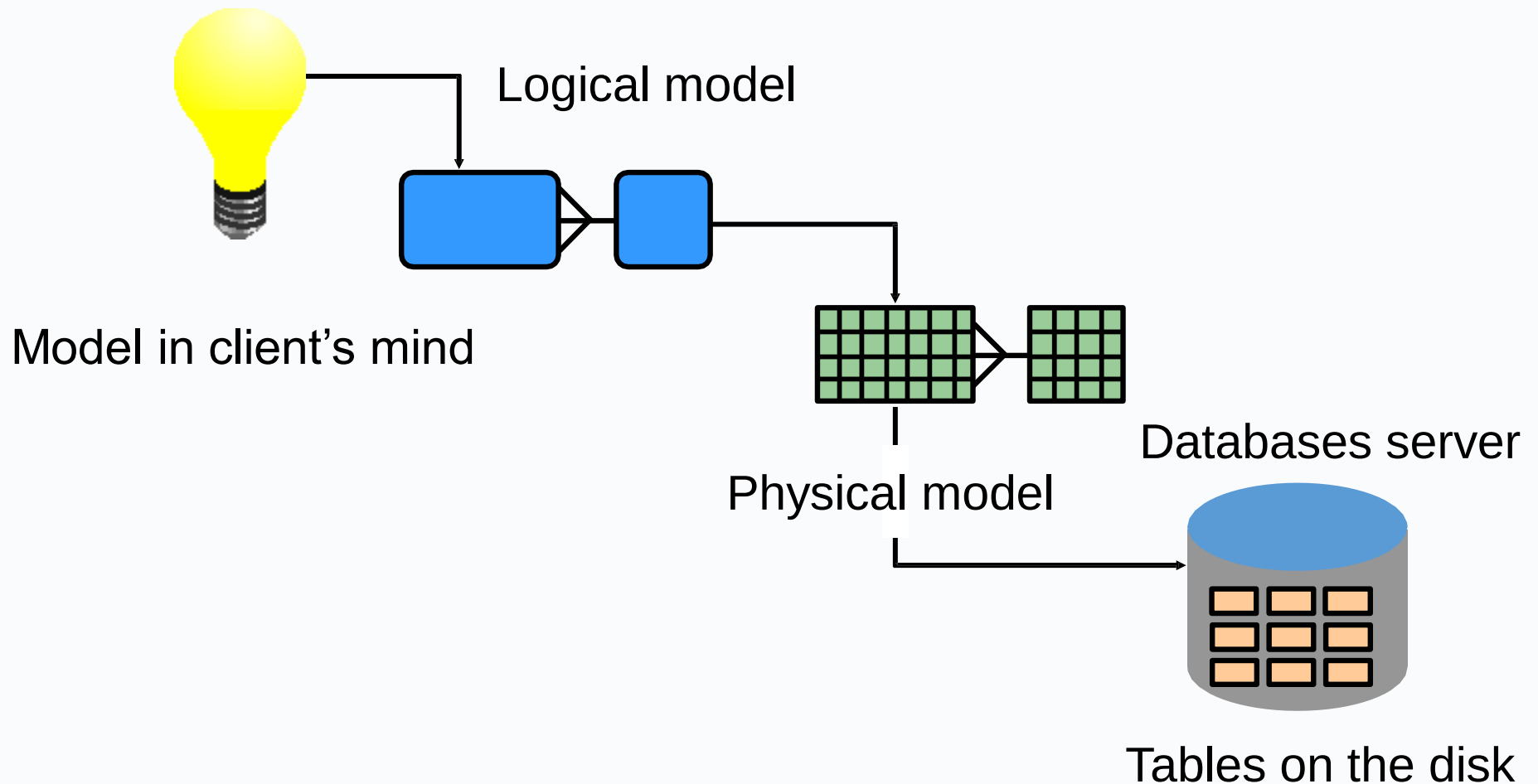


Normalisation

Dr Mariana Rocha
School of Computer Science

CPMU 2007 Databases 1, Lecture 9

Data model



Data design – start at the conceptual level

Figuring out what information purposes the database is to serve:

- What questions do the users need to be able to ask of the database?
- What pieces of data make up this information? What type of data is it?
- What things do these pieces of data describe?
- How the things we want to store data about can be linked together to provide the information needed?

The result is the **Logical Data Model**.

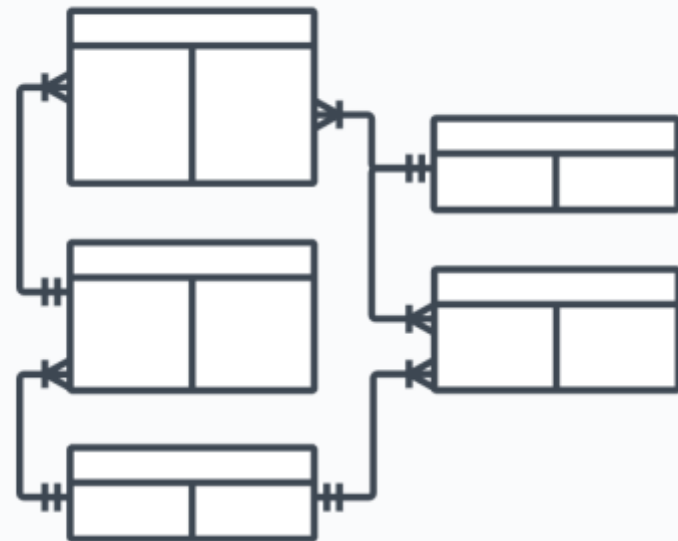
Entity Relationship Diagram (ERD)

An Entity Relationship Diagram (ERD) shows how different entities (such as people, customers, or other objects) relate to each other in an application or a database.

ERD components

An Entity Relationship Diagram is made up of many different components:

- Entity
- Attribute
- Relationship



Entity

- The entity is a thing or object of interest to the organisation being modelled. For instance, a person, a place, an object, an event, a concept
- Something about which data needs to be stored to fulfil some function
- Possible Entities:
 - Obvious physical things
 - Persons, places, objects
 - Transactions or events
 - Orders, Sales, Hospital Admissions, reservation
 - Plan, Schedule, Account, Course, Fund

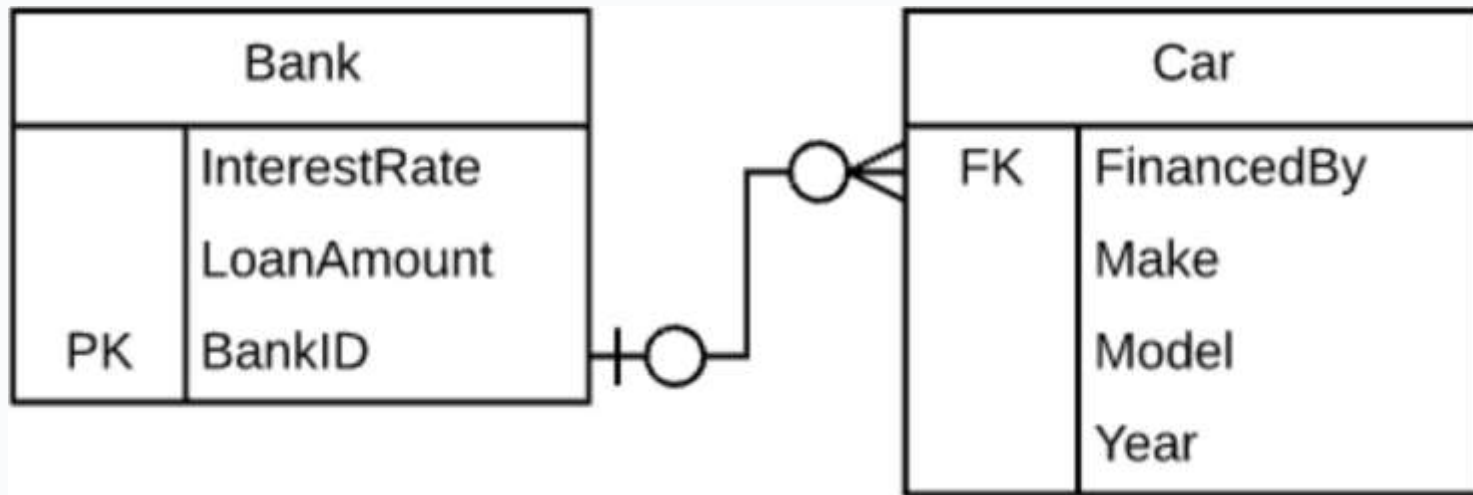
Attribute

- An entity is represented by a set of attributes, which are descriptive properties possessed by all members of an entity set.
- Attribute of an entity can be:
 - pieces of data we want to store about an entity
 - a descriptive property or characteristic of an entity
- We only want to record attributes that are of significance to the organisation
- The value of an attribute is the value of that attribute for a particular entity occurrence (or instance)

Relationship

- A relationship in an ERD defines how two entities are related to each other. There are several types of relationships that are represented on an ERD:
- One-to-one: One record of an entity is directly related to another record of an entity
- One-to-many: One record of an entity is related to one or more records of another entity.
- Many to many: Many records of one entity can be related to many records of another entity.

ERD



ERD - Structures and relationships

- So what are we doing when building an ERD?
- We are defining the structure of entities and the pattern of relationships between them
- When we implement that structure and pattern in a physical database then we can populate it with instances of the entity

ERD - Structures and relationships

- It is implementation independent of the DBMS
- You can implement the same conceptual model in any relational database (PostgreSQL, Oracle etc.)
- There may be small implementation differences, but the structure and relationships will be the same
- An Entity Relationship Diagram can be drawn at three different levels:
 - Conceptual
 - Logical
 - Physical

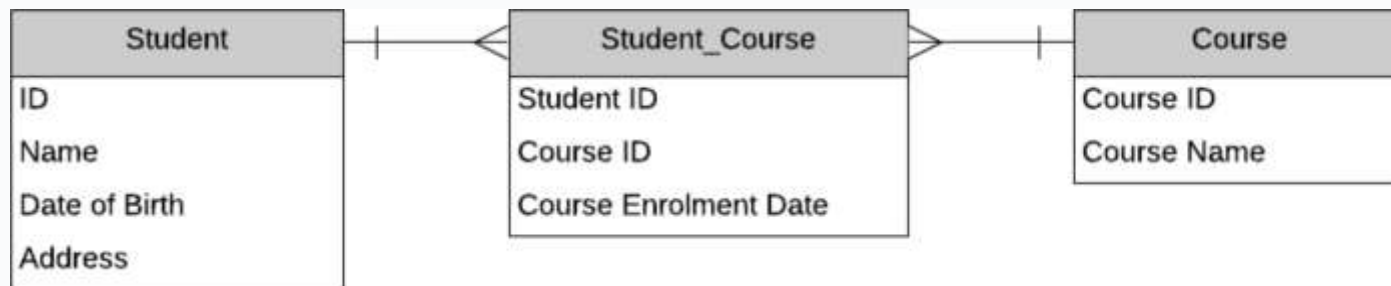
Conceptual model

A **high level** representation of the model showing the main entities (which may not end up as tables) mapped from design requirements



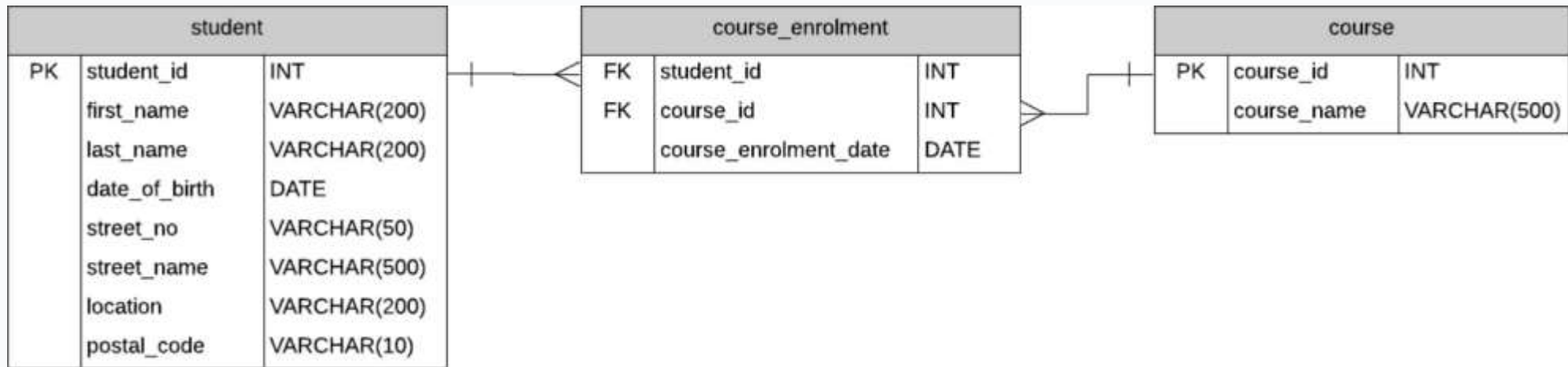
Logical model

A logical model is a **more detailed** version of a conceptual data model. Attributes are added to each entity, and further entities can be added that represent areas to capture data in the system.



Physical model

The physical data model is the **most detailed** data model in this process. It defines a set of tables and columns and how they relate to each other. It includes primary and foreign keys, as well as the data types for each column.



Physical model

Building the model right:

- Translating your logical model into the relational model
- Understanding the rules
- Eliminating redundancy

From which the physical database can be built

Data redundancy

- Duplication of data in separate tables
- Leads to waste and potential inconsistency
- How to overcome this?

Incorrect database design with data redundancy

Employer table

Employer ID	Employer Name	Address	Phone
10122	BeanTown Tours	105 State Street, Boston, MA 02109	617-451-1970
10125	Boston Harbor Excursions	75 Atlantic Avenue, Boston, MA 02110	617-235-1800
10126	BaySide Inn & Country Club	354 Oceanside Drive, Brewster, MA 02631	508-283-5775
10190	The Briar Rose Inn	105 Queen Street, Charlottetown PE C1A 8R4	902-626-1595
10191	Windsor Alpine Tours	14 Longmeadow Road, Laconia, NH 03246	603-266-9233
10198	Trudel Spa & Resort	40 Rue Rivard, North Hatley QC J0B 2C0	819-842-7783

Position table

Position ID	Employer ID	Employer Name	Position Title	Hours/Week
2040	10126	DaySide Inn & Country Club	Waiter/Waitress	32
2045	10122	BeanTown Tours	Tour Guide	24
2053	10190	The Briar Rose Inn	Host/Hostess	24
2066	10198	Trudel Spa & Resort	Lifeguard	32
2073	10126	Baside Inn & Country Club	Pro Shop Clerk	24
2078	10191	Windsor Alpine Tours	Ski Patrol	30
2079	10191	Windsor Alpine Tours	Day Care	35
2082	10125	Boston Harbor Excursions	Reservationist	40
2111	10126	BaySide Inn Club	Kitchen Help	32

data redundancy

inconsistent data

Key constraints – Foreign key

- An attribute of one table that represents the relationship between this table and another
- The attribute is the primary key of the other table
- Foreign keys provide a method for maintaining integrity in the data (called referential integrity) and for navigating between different instances of an entity.
- Every relationship between tables must be supported by a foreign key.
- Foreign keys attributes are indicated by the notation (FK) beside them on database models.

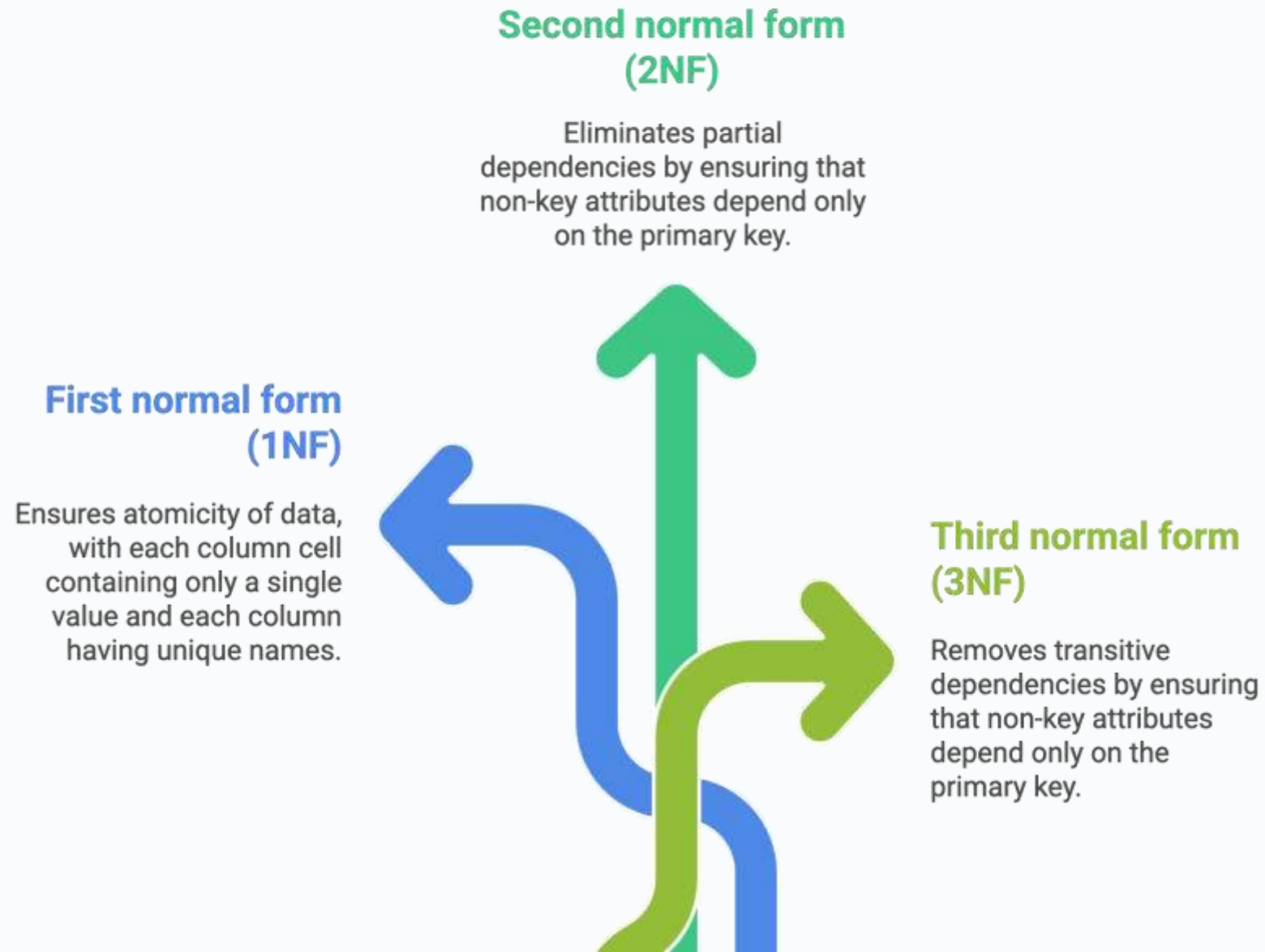
Normalisation

- A formal process involves reflecting on tables, attributes, and relationships to be created when developing a database most efficiently.
- The objective of normalisation is **to create relations where every dependency is on the key, the whole key, and nothing but the key.**
- Normalisation reduces **data redundancy** and increases **data integrity**, making it easier to save space on the server and maintain and access the data

Data anomalies

- Normalisation came as a solution to data anomalies, like data redundancy, which takes memory space and make it hard to manage the data.
- There are three types of anomalies:
 - Update anomalies: happen when the data is stored redundantly in the same table and someone when trying to update the data, **misses** some of the values.
 - Insertion anomalies: happen when inserting vital data into the database is not possible because **other data is not** there.
 - Deletion anomalies: happen when the deletion of unwanted information causes **desired information to be deleted** as well.

Normalisation



First normal form (1NF)

- Each cell should contain a single value (atomic value)
- All values in a column should be of the same type
- Each column should be uniquely identified

First normal form (1NF)

In the following table, we have cells containing more than one value.

InvoiceID	CustomerName	City	Product Line	ItemsBought	UnitPrice	Total	PaymentType	Date
INV001	Anna Byrne	Dublin	Bakery, Fruit	2.50, 4.00	6.50	6.50	Credit Card	2025-09-10
INV002	John Murphy	Cork	Dairy	3.00	3.00	3.00	Cash	2025-09-11
INV003	Anna Byrne	Dublin	Bakery	2.50	2.50	2.50	Cash	2025-09-12

How can we fix it?

First normal form (1NF)

InvoiceID	Customer Name	City	ProductLine	UnitPrice	Quantity	Total	Payment Type	Date
INV001	Anna Byrne	Dublin	Bakery	2.50	1	2.50	Credit Card	2025-09-10
INV001	Anna Byrne	Dublin	Fruit	4.00	1	4.00	Credit Card	2025-09-10
INV002	John Murphy	Cork	Dairy	3.00	1	3.00	Cash	2025-09-11
INV003	Anna Byrne	Dublin	Bakery	2.50	1	2.50	Cash	2025-09-12

Second normal form (2NF)

- For a table to be in 2NF:
 - The database is in its first normal form (1NF)
 - Every attribute should be dependent on the primary key

Second normal form (2NF)

- Here, our potential key = (InvoiceID, ProductLine).
- Attributes that depend only on InvoiceID move to an Invoice table.

Invoice table

InvoiceID (PK)	CustomerName	City	PaymentType	Date
INV001	Anna Byrne	Dublin	Credit Card	2025-09-10
INV002	John Murphy	Cork	Cash	2025-09-11
INV003	Anna Byrne	Dublin	Cash	2025-09-12

InvoiceItems

InvoiceID (PK)	ProductLine (PK)	UnitPrice	Quantity	Total
INV001	Bakery	2.50	1	2.50
INV001	Fruit	4.00	1	4.00
INV002	Dairy	3.00	1	3.00
INV003	Bakery	2.50	1	2.50

Third normal form (3NF)

- For a table to be in 3NF, there are two requirements:
 - The table should be in the second normal form.
 - All attributes should be determined (dependent) on the primary key and no other column. This means no attribute should be transitively dependent on the primary key. That happens when an attribute is not directly dependent on the primary key, but of another non-key attribute.

Transitive dependencies

- We still have transitive dependencies:
 - InvoiceID $\rightarrow$ CustomerName $\rightarrow$ City: city depends on the customer, not directly on the invoice.
 - (InvoiceID, ProductLine) $\rightarrow$ ProductLine $\rightarrow$ UnitPrice: the price depends only on the product, not on the invoice.
 - (InvoiceID, ProductLine) $\rightarrow$ (UnitPrice, Quantity) $\rightarrow$ Total: Total is derived from other non-key attributes (price $\times$ quantity).

Third normal form (3NF)

CustomerID (PK)	CustomerName	City
C01	Anna Byrne	Dublin
C02	John Murphy	Cork

ProductLine (PK)	UnitPrice
Bakery	2.50
Fruit	4.00
Dairy	3.00

InvoiceID (PK)	CustomerID (FK)	PaymentType	Date
INV001	C01	Credit Card	2025-09-10
INV002	C02	Cash	2025-09-11
INV003	C01	Cash	2025-09-12

InvoiceID (FK)	ProductLine (FK)	Quantity
INV001	Bakery	1
INV001	Fruit	1
INV002	Dairy	1
INV003	Bakery	1

Normalisation

- There are up to six normal forms available. It is recommended we try to normalise the database up to the 3NF.
- We can keep dividing the tables more and more – however, we need to always reflect on how necessary those divisions are

Normalisation

- Database normalisation:
 - makes the database more efficient.
 - prevents the same data from being stored in more than one place (called an “insert anomaly”)
 - prevents updates from being made to some data but not others (called an “update anomaly”)
 - prevents data not being deleted when it is supposed to be or data from being lost when it is not supposed to be (called a “delete anomaly”)
 - ensures the data is accurate
 - reduces the storage space that a database takes up
 - ensures the queries on a database run as fast as possible

What are normal forms?

- The process of normalisation involves applying rules to a set of data. Each of these rules transforms the data to a certain structure, called a normal form.
- There are three main normal forms that you should consider
- Whenever the first rule is applied, the data is in “first normal form”. Then, the second rule is applied and the data is in “second normal form”. The third rule is then applied and the data is in “third normal form”.

Normalisation – another example

StudentID	CourseID	ExamDate	ExamMarks	StudentName	StudentAge	Student Nationality	Course Description	Course ECTS
1	1	10/10/2019	56	Mark	21	Ireland	Maths	5
2	2	11/1/2019	56	Paul	19	Ireland	DB	5
3	3	10/2/2019	76	Mario	20	Spain	Java	10
1	3	10/2/2019	34	Mark	21	Ireland	Java	10
2	2	12/4/2018	65	Paul	19	Ireland	DB	5
2	1	15/3/2018	76	Paul	19	Ireland	Maths	5
1	3	10/4/2019	59	Mark	21	Ireland	Java	10

1NF

- If repeating groups are present, they have to be broken down into individuals' records and a primary key has to be assigned.
- In the example above, we need to find a key for the table.
- The composite key StudentID + CourseID + ExamDate is unique and it will represent our primary key.

<u>StudentID</u>	<u>CourseID</u>	<u>ExamDate</u>	ExamMarks	StudentName	StudentAge	Student Nationality	Course Description	CourseECTS
------------------	-----------------	-----------------	-----------	-------------	------------	------------------------	-----------------------	------------

2NF

- A partial dependency is a dependency between a non-key field (a field that is not part of the primary key) and a field that is part of the primary key.
- For instance, in our example, the student name depends on the studentID.
- The studentID is enough to tell us studentName. We do not need to know the full table key (StudentID+CourseID+ExamDate).

2NF

In the second normal form, the starting table most likely has to be divided in smaller tables so that every field in each table depends on the full primary key of that table. In this way the partial dependencies are removed.

2NF

StudentID -> StudentName, StudentAge, Student Nationality (if I know the studentID, I can know the StudentName, StudentAge and Student Nationality)

CourseID -> Course Description, Course ECTS (if I know the CourseID, I can know the Course Description and the ECTS (European credits)).

StudentID, CourseID, ExamDate -> ExamMarks (if I know the studentID+CourseID+ExamDate, I can know the marks the student got for that exam that day).

2NF

<u>StudentID</u>	StudentName	StudentAge	Student Nationality
------------------	-------------	------------	---------------------

<u>CourseID</u>	CourseDescription	CourseECTS
-----------------	-------------------	------------

<u>StudentID</u>	<u>CourseID</u>	<u>ExamDate</u>	ExamMarks
------------------	-----------------	-----------------	-----------

3NF

- In the third normal form we need to ensure that every attribute that is not the primary key must depend on the primary key and the primary key only.
- The set of the three tables in our example have no transitive dependencies and are already in the third normal form.
- We can stop the normalisation process here.

Process of data normalisation

- ELIMINATE REPEATING GROUPS
 - Make a separate table for each set of related attributes and give each table a primary key.
- ELIMINATE REDUNDANT DATA
 - If an attribute/column depends on only part of a multivalued key, remove it to a separate table.
- ELIMINATE COLUMNS NOT DEPENDENT ON KEY
 - If attributes/columns do not contribute to a description of the key, remove them to a separate table.
 - Everything should depend on the key the whole key and nothing but the key.